

All Models, Cross Validation & Grid Search — Simple Code Cheat-Sheet

Every snippet below runs on the same small weight/height → obese dataset (200 rows, 2 features), so you can copy-paste each block on its own.

0. Dataset + train/test split

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

n = 200
weight = np.concatenate([np.random.normal(68, 14, n//2), np.random.normal(85, 14, n//2)])
height = np.concatenate([np.random.normal(165, 7, n//2), np.random.normal(163, 7, n//2)])
y = np.array([0]*(n//2) + [1]*(n//2))
X = np.column_stack([weight, height])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
X_train = StandardScaler().fit_transform(X_train)
X_test = StandardScaler().fit_transform(X_test)
```

1. Logistic Regression

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression().fit(X_train, y_train)
print(model.score(X_test, y_test))
```

2. KNN

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
print(model.score(X_test, y_test))
```

3. Naive Bayes

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB().fit(X_train, y_train)
print(model.score(X_test, y_test))
```

4. Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier().fit(X_train, y_train)
print(model.score(X_test, y_test))
```

5. SVM

```
from sklearn.svm import SVC

model = SVC(kernel="rbf").fit(X_train, y_train)
print(model.score(X_test, y_test))
```

6. Cross Validation (k-fold)

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

scores = cross_val_score(LogisticRegression(), X, y, cv=5) # 5-fold
print(scores)
print("Average:", scores.mean())
```

7. Grid Search CV

```
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier

params = {"n_neighbors": [3, 5, 7, 9], "weights": ["uniform", "distance"]}
grid = GridSearchCV(KNeighborsClassifier(), params, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

8. Randomized Search CV

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.tree import DecisionTreeClassifier

params = {"max_depth": [2, 4, 6, 8, None], "min_samples_split": [2, 4, 6, 8]}
search = RandomizedSearchCV(DecisionTreeClassifier(), params, n_iter=5, cv=5, random_state=42)
search.fit(X_train, y_train)
print(search.best_params_, search.best_score_)
```

9. Bagging -> Random Forest

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
print(model.score(X_test, y_test))
```

10. Boosting -> AdaBoost / Gradient Boosting / XGBoost

```
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier

ada = AdaBoostClassifier().fit(X_train, y_train)
gb = GradientBoostingClassifier().fit(X_train, y_train)
xgb = XGBClassifier(eval_metric="logloss").fit(X_train, y_train)

print(ada.score(X_test, y_test), gb.score(X_test, y_test), xgb.score(X_test, y_test))
```

11. Stacking

```
from sklearn.ensemble import StackingClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

base_models = [("svm", SVC()), ("tree", DecisionTreeClassifier()), ("lr", LogisticRegression())]
model = StackingClassifier(estimators=base_models, final_estimator=KNeighborsClassifier())
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

12. K-Means Clustering

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, random_state=42).fit(X)
print(kmeans.labels_[:10])
```

13. DBSCAN Clustering

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

dbscan = DBSCAN(eps=0.6, min_samples=5).fit(StandardScaler().fit_transform(X))
print(dbscan.labels_[:10])    # -1 means "outlier / noise"
```

14. PCA (Dimensionality Reduction)

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

pca = PCA(n_components=1)    # reduce 2 features down to 1
X_reduced = pca.fit_transform(StandardScaler().fit_transform(X))
print(X.shape, "->", X_reduced.shape)
print("Variance kept:", pca.explained_variance_ratio_[0])
```